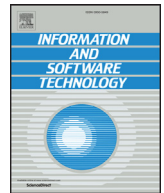




Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infosof

An industry experience report on managing product quality requirements in a large organization

Parastoo Mohagheghi^{a,*}, Mario Ek Aparicio^b

^a Norwegian Labour and Welfare Administration, Oslo, Norway

^b Miles Oslo AS, Oslo, Norway

ARTICLE INFO

Article history:

Received 3 May 2016

Revised 3 February 2017

Accepted 4 April 2017

Available online xxx

Keywords:

Product quality

Non-functional requirements

Agile

Experience report

Maintenance

Technical debt

ABSTRACT

Context: Eliciting, managing and implementing product quality requirements (in-short quality requirements) in a large organization can be challenging when many stakeholders are involved and projects run in parallel; sometimes with varying priorities with regards to quality. In this case from a public organization in Norway, the separation between business units and the IT-department and the legacy burden are additional factors that increase the complexity of requirement management.

Objective: This paper presents results and experiences from three years long work with quality requirements, starting from ad-hoc handling of quality requirements in separate projects to systematic work across projects with reusable sets of requirements and processes.

Method: We present how quality requirements are captured and classified, as well as changes to the agile software development process as a consequence of increasing focus on product quality.

Results: The ISO/IEC-25010:2011 standard is tailored for better context fit and is supported by concrete requirements and a methodology that covers the life cycle of software products in both greenfield and brownfield projects. In addition, the organization had to examine the current state of existing IT-capabilities in order to establish a quality baseline for future development, and develop shared vision and roadmaps for product quality.

Conclusions: In our experience, stakeholders prefer an iterative and lightweight approach in eliciting and refining quality requirements. The classification model and requirement lists are used as guidelines in requirement workshops. The developed terminology, updated templates and processes are reusable in projects and generalizable to different contexts, and are well adopted by the IT and business units.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

The term *Non-Functional Requirements (NFRs)* has been broadly used in the industry to cover the concerns other than functionality of an application or service. These concerns may cover how well an application or service performs and is experienced by users (such as its availability, usability, reliability and security), compliance with rules and IT-standards, or compliance with development constraints such as time, cost, resources, development processes and tools. Others have used the terms *product quality requirements* or *quality requirements* to cover the above concerns. In this paper, we use the term “quality requirements” for concerns other than functionality, while development constraints (such as time, budget and technical environment) are out of the scope of the work.

Quality requirements can be challenging to implement for numerous reasons [22]: First of all, unless developers and analysts proactively elicit quality requirements, the stakeholders might well believe them to be implicitly understood. Second, quality requirements tend to exhibit tradeoffs that must be carefully negotiated and resolved. Finally, quality requirements are often harder to measure and track than their functional counterparts.

This paper presents findings from the work to manage quality requirements efficiently across various organizational units and projects in a large public organization; i.e. The Norwegian Labour and Welfare Administration (NAV). In a large organization such as NAV, multiple projects are often running in parallel, a large number of stakeholders are involved and there are inter-relations between the results; thus requiring extensive communication and making tradeoffs when conflicts in goals or plans are detected. Huge effort is spent over time to elicit, anchor the “ownership”, and benchmark the level of performance. The success of work with quality requirements lies in involving stakeholders in a progressive way, developing sustainable requirement lists and processes that

* Corresponding author.

E-mail addresses: parastoo.mohagheghi@nav.no (P. Mohagheghi), mario.aparicio@gmail.com (M.E. Aparicio).

are reusable across projects and organizational units, an incremental improvement of the status of quality management support, shared responsibility for the results and open communication that enables learning.

This paper is an industry experience report that describes the context, problems that encountered during development, discussion of the methods that were developed and how we mean they solved the problems, and what lessons we learnt. By discussing the state of the art, we try to put the methodology and lessons learned in a broad context and make them useful for other practitioners as well as researchers.

The remainder of this paper is organized as follow: the context and motivation of the work is presented in Section 2 while Section 3 presents state of the art. In Section 4, sources and types of quality requirements are discussed and Section 5 presents processes developed for managing quality requirements. These processes, the classification model and requirements baselines are elements of the methodology for managing quality requirements. Section 6 discusses the impact of the work on practice and includes examples from different perspectives; thus evaluating and validating the methodology. In Section 7, we present experiences from workshops with requirement owners. The contributions of this work relative to the state of the art, the validity of the results and key lessons from our work are discussed in Section 8. Finally, Section 9 presents conclusions and directions for future work.

2. Background and motivation

NAV administers a large proportion of welfare benefits and social security schemes in Norway. These include almost 50 types of benefits that cover different stages of life of citizens such as parental leave, unemployment benefit, sickness benefit, disability pension and retirement pension. With over 2.8 million users, NAV pays out one-third of the Norwegian national budget in benefits. Over 400 IT applications provide services to citizens as well as to 19,000 employees of NAV who manage the benefits. NAV emerged in 2006 as a result of a merger of three organizations responsible for different benefits; resulting in a large organization with a burden of legacy IT systems from different domains and built over time using multiple technologies. Many new online services have been developed since 2006. However, the IT landscape still includes many legacy systems, the oldest from 1978.

With the vision of offering more user-friendly and available services, NAV is heavily modernizing its services and IT-portfolio, moving from paper-based processes with manual processing to self-service and online services where most of the processing should happen automatically by codifying rules to check eligibility and enable disbursements. The IT-department in NAV with over 570 employees manages IT-services and applications across the organization. A *service* in this context refers to what is offered to end users; external as well as internal. An *application* is a component or system that offers the service or is used in the process of offering the service. In the remainder of this paper we use the term *IT-capability* to generally cover services and applications. The directorate in Oslo administers NAV and the organizational units in the directorate (called business units in this paper) are responsible for defining, prioritizing and financing changes to be done to IT-capabilities using a portfolio management process. When participating in development, they have the role of *product owners* [35] and are responsible for refining requirements, prioritizing items in project backlogs and accepting the results.

New development and major updates are mostly handled by initiating projects. NAV's internal resources (from business units as well as the IT-department) manage the projects, specify requirements and are responsible for quality assurance and acceptance of the resulting software products. They also participate in

software development in some specific roles and have the overall responsibility for operation and maintenance. The practice by now has been to charge external contractors with most software development and technical maintenance based on contracts with NAV. This is however changing gradually and NAV is taking more responsibility for development and maintenance of IT-capabilities.

NAV's IT-department manages the overall software architecture and processes for development and maintenance. Since 2012, the development process is an agile methodology, which is an adaptation of Scrum [35]. Some extra activities and roles are added for using agile on a large scale and integrating it with other processes such as the portfolio management process. Examples of extra activities are managing a product backlog across projects and evaluating consequences of proposed changes by the central software architecture team.

NAV started a large modernization project in 2012 with the goal of supporting new political reforms as well as offering more self-services, improved management of benefits and more automation. Business units specified the new IT-capabilities that were later assigned to different development tracks and contractors. Quality requirements were, however, only listed in the initial project specification document and were not part of epics or user stories. Recognizing the need to specify the qualities of services, the modernization project asked development and maintenance teams for input and gradually collected a list of quality requirements. This list gathered over 1000 requirements classified using the ISO/IEC 25010:2011 standard [26]. The high number of requirements was by itself a challenge. Additionally, many requirements were not adequately specified. We refer here to the definitions of a SMART requirement [17]:

1. *Specific*: the quality requirement is clear and unambiguous.
2. *Measurable*: the quality requirement is measurable so that it is possible to know if it has been delivered.
3. *Relevant*: the quality requirement is relevant for the types of IT-capabilities the organization has and the desired qualities of these in the future.
4. *Attainable*: the quality requirement is realistic and attainable.
5. *Time-bound*: it should be a target date for the quality requirement or in our case often a road map for improving the state.

Many quality requirements in the list were not measurable and were not assigned to deliverables and release plans. Some quality requirements would drive cost and complexity excessively. In addition, business units meant that they were not adequately involved in the elicitation process and were especially concerned that compliance requirements were missing. A public organization must adhere to a host of laws and regulations such as privacy requirements and public archiving. Finally, the list mixed quality requirements for IT-capabilities to be delivered (product quality requirements) with other types of quality requirements such as process quality requirements.

Following the agile development process, functional requirements are defined in *epics* and *user stories* and define *what* an IT-capability is expected to do. Quality requirements on the other hand describe *how well* those functions are accomplished. User stories are not part of Scrum but are one technique among many to specify product backlog items. NAV's agile development process did not originally include explicit guidelines on how quality requirements should be addressed.

With the recognition that the modernization project needed support for managing quality requirements, the IT-department started a small project to improve the organization's maturity on quality requirements; called here for the *QR-project*. The QR-project started in November 2012 and was initially planned to be a short task force with focus on the elicitation and specification

steps of requirements. However, the endeavor was more difficult than we realized and the project was running for 18 months for several reasons. First, the size and complexity of the organization created challenges. There are many stakeholders that needed to be included and informed. Stakeholders include business units that play the role of customer, end users, developers, maintenance and IT-operations in the IT-department, project managers, government regulators, and in some cases external agencies that exchange data with NAV. Some quality requirements even had implications for other projects than the modernization project. Further, we realized that the development process needed to be extended with new roles and responsibilities across the organization. In fact, the QR-project triggered activities across the organization that now happen on a continuous basis.

The QR-project had between 2 to 5 participants in periods (none of whom were fully allocated to the project) while it involved many others to work with specific requirements. The authors of this paper have been leading the QR-project from its beginning and present here the achievements as well as experiences and lessons learned. The authors have also followed the work after termination of the QR-project by supporting different projects and maintenance teams. The achievements of the project may be described as a classification model for quality requirements, lists of requirements with benchmark data, experiences from requirement elicitation workshops, and the developed process for managing quality requirements that is integrated in the agile development process.

3. Literature review

This section presents state of the art on the subjects of requirement elicitation with focus on quality requirements, classification models for quality requirements and large scale agile development methods.

3.1. Requirement elicitation

Requirement elicitation has been subject of extensive research and over time, several requirement elicitation techniques have been proposed. Examples are interview-based methods, prototyping, specification by example, observing users, task analysis and requirement workshops [36,42]. The distinction between functional and quality requirements exists in most requirement classification methods; see for example [26,30,36]. Several studies have shown the significance of quality requirements in software projects as well [8,16,23,30,36]. Capturing qualities can either be done using techniques for capturing functional requirements or other more specific approaches, such as the Software Engineering Institute's Architecture Tradeoff Analysis Method (ATAM) [4] and Quality Attribute Workshops (QAW) [5]. Other techniques such as Impact mapping [2] may be applied as well; which is a technique that helps organizations describe their goals, communicate assumptions and develop roadmaps to achieve the goals. Conflict identification and resolution is part of requirement elicitation since stakeholders' interests and priorities are different. Conflict resolution may specially be challenging for quality requirements since dependencies between quality requirements are often complex and some requirements may even not be compatible with others. To identify which stakeholders to notify of a potential conflict, a stakeholder / quality-requirement relationship map may be useful that shows which stakeholders are interested in which quality requirements [25]. Results of a case study on 15 digital library systems in university environment presented in [25] showed that in case of conflicts of interests, a tendency exists to gravitate to satisfactory solutions and not necessarily optimal solutions unless the issue is at the core of a stakeholder interest.

There has also been focus on capturing quality requirements early in the development process since some requirements drive architectural decisions; thus the term "Architecturally Significant requirements (ASRs)" has emerged [11,12]. Chen et al. define ASRs as requirements that have a measurable impact on a software system's architecture [11]. "Significant" indicates the high cost of change. ASRs have wide impact, target tradeoffs, are constraining and are often difficult to achieve. In practice, many projects fail to adequately explore quality concerns. For example, Franch and Carvalho conducted a survey of ASR elicitation techniques in 13 different software projects and found that in many cases projects did not specify any such requirements [21].

Interesting research is presented in [7] where Bellomo et al. have analyzed ATAM data reported in publications from 1999 to 2013. Quality requirements most presented in the data were related to *modifiability*, *performance*, *interoperability*, *availability*, and to a lesser extent *usability*. These characteristics dominate over the years despite changes in the technical environment for software systems. In more recent studies, *deployability* has emerged as an important characteristic.

3.2. Classification of quality requirements

Many software quality models are developed during years for classifying quality requirements [9,22,23,31,39]. The first work dates back to the late 1970 s, when Boehm described quality characteristics and their decomposition [9]. In [31], Miguel et al. provide an overview of software quality models and shows the evolution of quality models from the Mc Call first model in 1977 until 2013. Software quality models may be classified in different ways. Klas et al. distinguish three categories of models [27]: 1) The level of general public use or specific domain, 2) the organizational level that focuses on satisfying the interests of a specific organization, and 3) the project level that applies to a specific project to ensure quality. This work has an organizational perspective that extends beyond single projects.

3.3. Quality requirements in an agile context

Traditional requirement elicitation approaches are often considered too heavy-weight by agile practitioners. However, "a healthy focus on architecture is not antithetic" to agility [1] and late discovery of quality requirements may lead to the wrong architecture being implemented and consequently the need for costly refactoring. Concerns about long-term deterioration of quality in large-scale agile projects have renewed the agile community's interest in software architecture [7]. The challenge is in balancing up-front design and emergent architecture [19]. Scott Ambler proposes some kind of middle ground [3], in which architectures are sketched in early phases of the project; thus planned and delivered incrementally.

Since many agile methodologies such as Scrum often can be seen as toolboxes instead of full-fledged frameworks, there are a host of various implementations. Practitioners of agile methods have proposed several ways to specify quality requirements such as defining user stories for quality requirements, adding them as constraints or as acceptance criteria to other user stories, or including quality requirements in the Definition of Done (DoD); see for example [34]. However, there has been less focus on how to capture quality requirements in the first place when applying an agile method. Product owners often consider qualities to be simply in place and are not concerned about the costs, tradeoffs and architectural constraints. In addition, many projects do not start from scratch. They either update existing functionality or are developed using existing services, platforms and technologies. Thus they should take the characteristics of so-called "legacy" into

account when specifying project requirements. Finally, the elicitation of functionality, quality and software architecture are often in practice intertwined and some refinement is necessary [16].

In the context of scaling agile methodologies, a few frameworks have emerged. We discuss here the Scaled Agile Framework (SAFe) [34], Large-Scale Scrum (LeSS) [28] and the Disciplined Agile Delivery (DAD) [15]. In the context of NAV, it is more suitable to draw comparisons to these large-scale frameworks instead of basic methodologies such as Scrum. As mentioned earlier, many agile methodologies have the shortcoming that they lack proper guidelines on how to handle quality requirements. Since we believe proper handling of quality requirements is essential in large environments, we expect that the prominent large-scale agile frameworks provide guidelines for handling quality requirements in an agile setting.

SAFe applies NFRs in connection to what they call *system qualities*. NFRs are classified as constraints that restrict freedom of software design and implementation. In the SAFe metamodel, NFRs are modelled as constraints on backlog items (epics and user stories). SAFe introduces three software development levels; i.e. the Enterprise, Program and Team levels; and adds that NFRs have relevance at all the levels. This means that a wide variety of roles at different levels must work with defining, refining and implementing NFRs; such as product owners, enterprise architects, systems architects and developers. NFRs are typically revisited as part of the Definition of Done for each iteration, program increment or release. DAD; developed by Scott Ambler; is a hybrid framework that builds upon many other methods. At the enterprise level, it adopts SAFe and Enterprise Unified Process (EUP) [20], which refers to the application of agile across an organization.

In LeSS, requirements are categorized in “Requirement areas”, which is a view of the product backlog that collects related requirements. A requirement area may be functional or non-functional (such as Performance) and collects feature teams that are organized around customer requirements. We deem this approach inappropriate for cross-cutting quality requirements that cannot modularized out.

4. Sources and types of quality requirements

NAV has over 400 applications to maintain and a high pace in IT development. One critical task for specifying quality requirements is to identify *stakeholders*. Stakeholders include the *requirement owners* who are responsible for monitoring and maintaining the requirements on the long term. Other stakeholders, such as product owners may change the quality state of IT-capabilities. For example, the business unit responsible for customer interaction defines the availability requirement for online services and is thus the requirement owner. The operations unit in the IT-department is responsible for providing an infrastructure that supports this requirement as well as monitoring the availability of online services. A project that develops a new online service should consider the availability requirement and provide the agreed level of service quality. Different stakeholders must agree on the level of quality and balance their interests.

We started our work by analyzing quality requirements in the modernization project and assessing their status. This stage covered two activities:

- A bottom-up analysis of all collected quality requirements to identify areas of interest.
- A top-down analysis of requirements stated in the “Project objective document”.

In total 22 “quality attributes” were identified at this stage. A *quality attribute* in this context refers to a high level need for a particular quality that should be underpinned by more refined

and quantified requirements. We further realized that some quality requirements applied across multiple projects while others were specific project’s particular software components. Quality requirements were therefore separated into two groups:

- Quality requirements that apply for most projects and that act as baseline when reusing existing IT-capabilities or developing new ones. These quality requirements are gradually collected in the so-called *Basic lists*.
- Quality requirements that apply to a specific IT-capability to be developed or modified in a project. Product owners in collaboration with requirement owners must specify which level of quality is required for each IT-capability. These are further detailed into sub-attributes (quality requirements) and collected in *solution-specific lists*.

The basic lists include requirements covering:

- Architectural constraints and standards for interoperability.
- Operations requirements regarding infrastructure to be used, backups, workload and availability.
- Requirements related to dialogue with users, such as availability, user-friendliness, response times and accessibility for users with disabilities.
- Security and privacy requirements.
- Maintainability requirements such as code quality, modularity and test coverage.
- Requirements related to compliance to laws and regulations.

Fig. 1 shows an overview of quality requirements and their sources, as well as a few examples of quality requirements. New projects must consider the basic lists and either accept the requirements or negotiate for exception with requirement owners. Requirement owners are business units or units in the IT-department. It is however important to identify risks and consequences (for users, maintenance and future upgrades) in case of exceptions. They should in addition consider the quality state of legacy systems in case of reuse and specify their own solution-specific quality requirements. Requirement ownership is transferred to a business unit or the IT-department when the project is finished.

All quality requirements from all sources are classified using the ISO/IEC 25010 standard. There are other standards and frameworks for classifying quality requirements as well, as discussed in Section 3.1. However, we chose the ISO/IEC 25,010 standard since it was already adopted by the modernization project and was well known in the organization. The standard covers eight definitions of *product quality characteristics* as shown in Fig. 2 and marked by gray color. Each quality characteristic in the standard has a number of *sub-characteristics*. We used these as-is with two exceptions:

- The ISO/IEC 25010 standard defines *Reusability* as a sub-characteristic of *Maintainability*. We changed this to *Reuse* and divided it into two aspects: *Reusability* as the degree to which an asset can be used in building other assets, and *Degree of Reuse* as the degree an application is built using other assets.
- For *Security*, we followed the classification model defined by NAV’s information security staff. Security has the sub-characteristics *Confidentiality*, *Integrity* and *Non-repudiation*.

However, we realized that the ISO/IEC 25010 standard does not cover two areas of interest for NAV:

- *Compliance to laws and regulations*: all decisions regarding benefits, calculations and supporting activities such as payment must follow laws and regulations defined by the government. For *Compliance*, we identified one sub-characteristic, which is the *Degree of compliance with stated laws and regulations*.

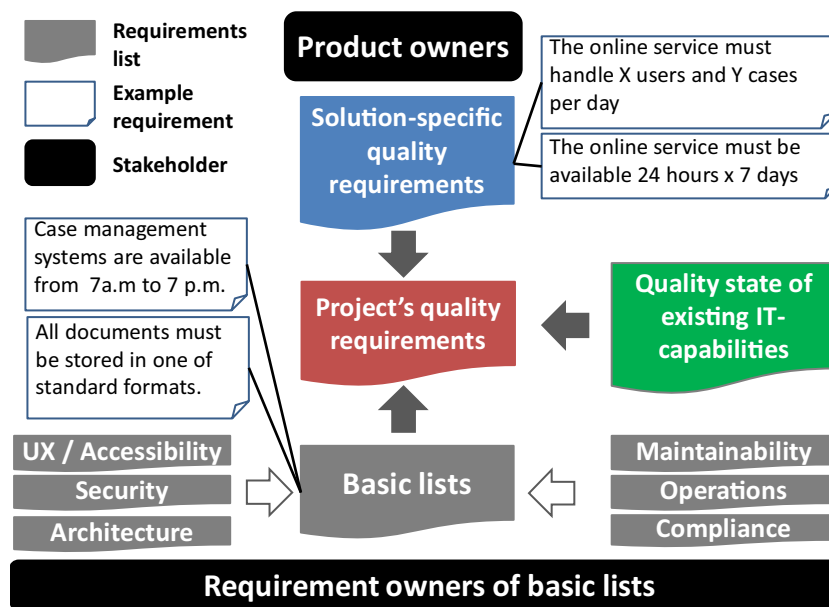


Fig. 1. Types of quality requirements that must be considered by a project.

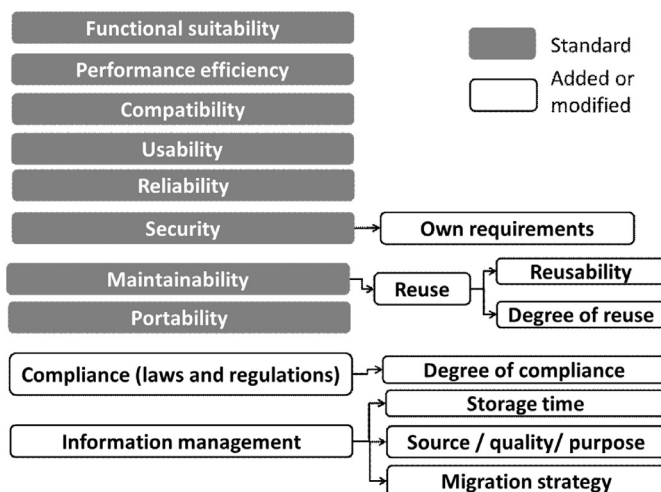


Fig. 2. Tailoring the ISO/IEC 25010 model.

- **Information management:** it covers the quality of data involved in the case management processes and managing these data. It has three sub-characteristics: *Storage time* – each information object should have defined time for validity and storage, *Source /quality /purpose of data* – an information object that is produced or processed should have defined source, known quality, and a registered purpose of use, and *Data migration strategy*– the need for conversion or migration of data.

There are standards within the ISO/IEC 25000 framework that already cover information management, or more precisely *Quality of Data Product* (ISO/IEC 25012). However, we wanted to create a quality model that reuses terms already established in the organization, while ISO/IEC 25012 may be applied in the future. Compliance and Information management were therefore added as extensions to the classification model.

In order to make the classification model more understandable and user friendly, we detailed some of the sub-characteristic to a third level and provided concrete examples. For example,

Maintainability has a sub-characteristic which is *Analyzability*. We added examples at the third level such as:

- **Monitoring:** each IT-capability should provide possibility for monitoring.
- **Documentation:** it should be specified which documentation to be delivered for an IT-capability.
- **Time needed to analyze and correct a bug:** this requirement is always specified in contracts.

Other examples are:

- **Functional appropriateness** has two specific requirements at level 3:
 - *Process automation*; defined as “the extent the IT-capability supports process automation”. It is necessary to have an optimal balance between automatic processes and manual tasks to avoid unnecessary cost and complexity.
 - *Functional support*; defined as “the extent the IT-capability functionally supports the user in performing manual tasks”.
- **Operability** has one specific requirement at level 3:
 - *Transparency*; defined as “the extent the IT-capability gives the user sufficient and correct information to understand the process and tasks he/she is performing through the user interface. Legal implications, rights and duties should be visible and understandable”.

Examples are meant as guidance and the users of our model are free to add further refinements at the third level.

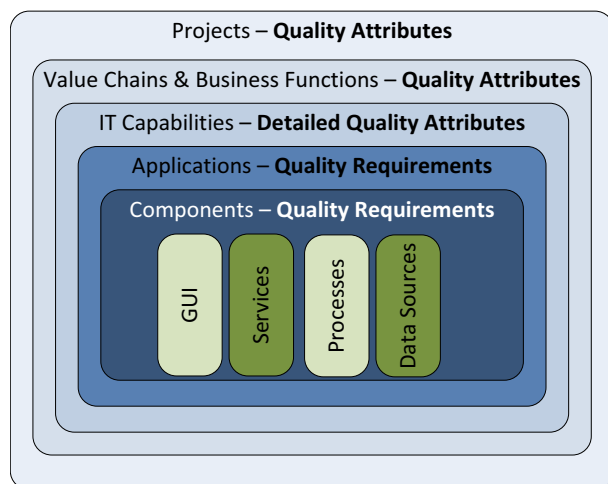
The focus on quality requirements triggered the business side to work systematically with compliance requirements. These requirements come from hundreds of pages of legislative documents. Based on these documents, the business unit owning these requirements has extracted quality requirements regarding document storage, information access, transparency of information and other requirements important for IT-capabilities in the public sector. The lists save new projects a lot of effort and makes sure that IT-capabilities are compliant with regulations.

The feedback has been very positive and the model and example requirements have been widely adopted as reference lists in requirement elicitation workshops. The developed model has been stable since developed for two years ago. The model is translated into Norwegian, which is the working language in the organization.

Table 1

Excerpt from a typical requirement list for a project.

Req. set	Req. area	Req. ID	Description	Quality characteristics	Source
Solution X	Business Requirement	P1	The online service must be able to handle X users and Y cases per day.	Performance / Capacity	Solution Specific
Solution X	Business Requirement	P2	The online service must be available 24 h per day and 7 days a week.	Reliability / Availability	Solution Specific
Operations	Client platform	O3	Citrix version <i>n</i> must be used by applications.	Portability	Basic List
Architecture	User interaction	B7	User interaction and business logic should be decoupled.	Maintainability–Modularity	Basic List
Operations	Delivery	DB6	Large databases should be partitioned.	Performance /Capacity	Basic List
Operations	Delivery	U4	User documentation should be updated and be accessible.	Maintainability / User support	Basic List
Online service	User Experience	U1	The language must be adapted to user group and the expected level of knowledge.	Usability / Accessibility	Basic List
Online service	User Experience	U2	At least <i>n</i> % of users must be able to use an IT-service without getting help.	Usability / Operability	Basic List

**Fig. 3.** Quality requirements are specified and managed at multiple levels.

Quality requirements in the basic lists are revised and simplified in multiple rounds to make them understandable for stakeholders. Requirements are maintained and improved on a continuous basis and baseline data is gradually added to the requirements. These are important measures of the current level of quality. Before starting our work with quality requirements, new initiatives spent considerable amount of effort trying to understand the existing state. It was difficult to define improvement goals and acceptance criteria without any baseline data. One contribution of this work has been to highlight the importance of having a point of reference and communicating this across the organization so that risks of non-compliance and low quality are identified. The basic lists are all posted on the architecture portal that is owned by the IT-department's architecture team and are accessible by everyone. The team is responsible for maintaining the lists in conjunction with requirement owners. The architecture team also assists projects in evaluating the impacts of quality requirements and feasibility of new requirements. Table 1 contains some examples of quality requirements and shows how these are documented.

Finally, we add that quality requirements must be considered on multiple levels; from project level and business functions to services, applications and systems components. On the project level and with relation to relevant business functions one works with high-level quality attributes while actionable quality requirements are used on the technical implementation level. Fig. 3

depicts the multi-level view; from desired quality attributes and the pertinent business functions to concrete quality requirements with acceptance criteria that will be applied to applications and underlying technical components. All these levels need to be communicated well between stakeholders in order to have a complete view of the quality of the resulting software.

5. Processes

While having lists and examples of quality requirements is useful, they should be accompanied by processes that describe the steps to follow in order to assure coverage and fulfillment of requirements. With *coverage* we mean that quality requirements that are important for achieving business goals should be discovered in time. For *fulfillment*, the consequences should be evaluated and adequate resources be allocated to develop a solution that meets the requirements. In this section we describe the processes that are developed for managing quality requirements during development and maintenance.

5.1. Managing requirements in projects

The overall process for managing requirements in a project consists of five main stages as depicted in Fig. 4.

The activities are defined to be:

1. *Establish*: The goal of this step is to identify the relevant quality requirements, clarify the existing state of quality and decide whether it is good enough or should be improved. For a new IT-capability, the goal is to get an overview of the constraints that should be considered as well. This step thus consists of two parts.
 - The basic lists are reviewed to extract relevant quality requirements for a project.
 - The project specifies additional quality attributes and requirements that are most important for the IT-capabilities to be developed. These are captured in epics and user stories or as acceptance criteria.
2. *Detail*: The identified quality attributes and requirements are detailed; clarifying which quality requirements apply to which deliverables and how requirements shall be verified. This stage involves architectural analysis and evaluation of the consequences and costs by domain experts, software architects and others such as maintenance, operation, test and security experts. This step is done in negotiation with the contractors / developers, letting them comment on which quality requirements



Fig. 4. Activities to be performed for managing requirements in projects.

they deem difficult or impossible to fulfill. Quality requirements specific for the project are either added to functional user stories or defined in separate ones.

3. *Develop and test*: IT-capabilities are developed and code reviews, unit tests and integration tests are performed.
4. *Verify*: quality requirements are verified in acceptance tests or in design reviews by product owners.
5. *Reporting and handover*: IT-capabilities are handed over from projects to the organization for maintenance. Compliance reports are made so that stakeholders can see how their quality requirements are handled. Requirement owners update the baseline data in basic lists to reflect the state of quality after handover.

This process is a typical process for requirement management except for the review of basic lists in the beginning, clarifying the state of existing IT-capabilities and the handover step which is important from a maintenance point of view. Large organizations typically have a portfolio of applications that make the basis for any future modification and information should not be lost in handovers. Usually requirement management processes cover phases until verification and do not address the maintenance stage. In most cases, the problem of a new project is not about identifying new requirements but evaluating the impact on existing applications and services. When the quality of these applications is made explicit, uncertainties are avoided and problems are discovered early. The basic lists are reviewed together with the contractors in the contract phase and compliance is verified during handovers. Additionally, the basic lists are an important tool in baselining and supporting prospective improvement initiatives as described in the next section.

The activities of evaluating the quality requirement lists in Step 1 and handover in Step 5 may be quite lengthy and require many hours of workshops with different stakeholders. Fortunately, the lists are reused in subsequent projects.

5.2. Managing quality requirements between projects

Between projects, the basic lists are maintained by the IT-department and requirement owners. It is worth noting that fulfillment of quality requirements represents the technical state of applications and services and is used to define possible *technical debt*. The state is further used to justify spending resources to fix “unfulfilled quality requirements”. The term technical debt is relatively broad and somewhat ambiguous. NAV has adopted a definition that fits very well with the work with quality requirements. In [37], technical debt is defined as a metaphor that refers to the consequences of poor software development. In practice, it refers to the gap between the current state of a system and the state the system would be “ideal” for the environment it acts in. Aspects that can cause technical debt are failures, unimplemented functionality that were in the scope, under-performance or poor documentation. In effect, unfulfilled quality requirements constitute technical debt, and technical debt affects the state of quality. Since the IT-department now has an overview of the quality, it is possible to reduce further growth of technical debt that is due to under-performance. New projects are required to deliver “at least the same quality as before” and document any issues that should be improved. Requirement owners then decide whether these

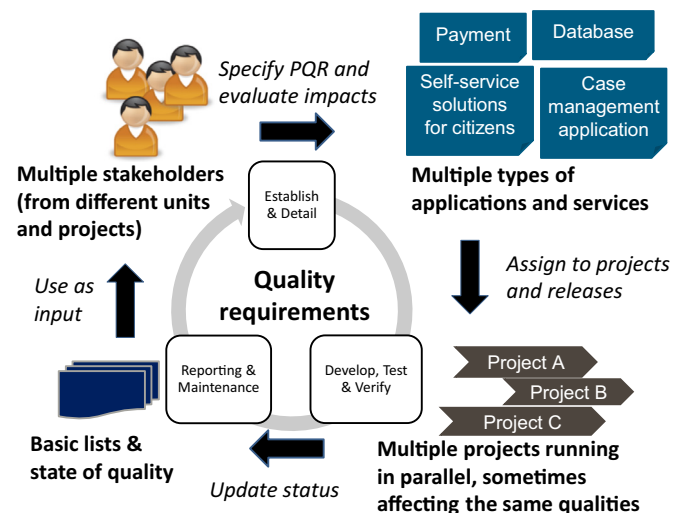


Fig. 5. Managing quality requirements is an iterative process.

issues should be added to the product backlog as technical debt. Not all previous technical debt can be removed but an evaluation of risks and costs is performed for major applications. Issues that are considered as critical are defined in the shared product backlog and gradually solved; either in projects or as maintenance tasks.

The whole lifecycle of negotiation, development and updating the status of quality requirements is illustrated in Fig. 5. Requirements covering the critical qualities are collected and handed over to projects and developers in the *Establish* phase. Deviations to the approved quality requirements should be discussed with requirement owners. Defining a project's quality requirements is an activity that involves multiple stakeholders (requirement owners in the organization, product owners in projects, and developers / contractors). When a project is finished, the new or modified IT-capabilities are handed over to NAV for operation and maintenance and quality status is used as baseline for future initiatives. Unfulfilled requirements (as input for evaluating technical debt) are recorded and reported back to the organization.

6. Validation of the approach

Quantitative evaluation of a process change can be difficult and costly. However, we can study the impact of the new process on practice in several increments and projects. With enough examples, we can also evaluate whether the approach can be generalized to a broad set of projects or contexts.

The architecture team in NAV has reviewed and accepted the tailored ISO/IEC 25,010 model as NAV's model for quality requirements. Quality requirement lists together with detailed descriptions of quality characteristics are published on the architecture internal webpages (*architecture portal*) and are accessible to everyone. Other lists worked out by the projects or maintenance teams refer to these definitions, thus achieving consistent definitions across the organization, which was missing when we started our work. The collection of basic lists includes at the moment around 250 quality requirements that should be evaluated by all projects and deviations should be documented. The volume is thus

Table 2

The state of fulfillment of baseline quality requirements of a main application.

Req. set	Req. area	Req. ID	Description	Quality characteristics	How to verify	Fulfilled	Action
Operations	Client platform	O3	Citrix version <i>n</i> must be used by applications	Portability	Test the application in Pre-operations and after 2–3 days in operation	NA	
Architecture	User interaction	B7	User interaction and business logic should be decoupled	Maintainability-Modularity	Guideline for design	Partly	Some existing services have tight coupling to DB. Fulfilled for Java code.
Operations	Delivery	DB6	Large databases should be partitioned	Performance /Capacity	Contractor must document if a DB is larger than 4 GB	Yes	
Operations	Delivery	U4	User documentation should be updated and accessible	Maintainability / User support	Contractor must deliver updated documentation	Partly	User documentation must be updated in 2016

Table 3

Two requirements defined by business units for online services.

Type of application	Quality characteristic	Req. ID	Description	How to verify	General (✓) or solution-specific (P)	Action
Online service	Usability / Accessibility	U1	The language must be adapted to user group and the expected level of knowledge	User testing	✓	User interaction testing team monitors the state
Online service	Usability / Operability	U2	At least <i>n</i> % of users must be able to use the IT-service without getting help.	User testing. Test objects must cover a broad range of users.	P	The percentage of users is defined by each project. The user interaction testing team must be asked for advice. See examples (reference)

reduced from over 1000 requirements identified initially to ¼, which is manageable. In addition, the requirements are concrete. The largest groups of requirements are related to the usability (90 requirements), security (50 requirements) and maintainability (50 requirements).

This tailored classification model and the basic lists are used in requirement workshops to help identifying relevant requirements. Every project and major release has an own space on the internal webpages where quality requirements are documented. Discussing these requirements with developers is a must and different types of tests and scenarios should be specified in order to verify the quality of services. The lists are also used to evaluate the status-quo of major systems. An example is a large legacy system that manages certain types of benefits. Around 190 quality requirements in the basic lists are identified to be relevant for this system. Of these, 100 requirements are fulfilled, 35 are partly fulfilled while 55 are not fulfilled. Some of these 55 requirements cannot be fulfilled due to the choice of technology or other restrictions while 21 are planned to be followed up. For another system, 71 of requirements in the basic lists are considered as relevant. From these, 40 requirements are fulfilled, 5 are partly fulfilled and the remainder 26 requirements are not fulfilled.

The templates for specifying epics and user stories are updated as well and include a section to specify quality attributes (similar to Tables 2 and 3). For example, an epic in a product backlog for allowing users to view their documents includes:

- 11 stakeholders; five stakeholders in the IT department (security, document management, etc.) and six on the business side from different units (legal section, user dialog, etc.)
- Three identified risks related to the reuse of existing functionality, managing load and performance
- A few pre-conditions
- Description of the functionality with 21 scenarios for usage

- Value for NAV (saved effort due to fewer meetings and telephone calls, more transparency) and users (higher satisfaction, saved effort and the increased possibility to contribute to their case). Value is measurable in money and improved quality.
- Existing situation and why it should be improved
- Three actors and the desired process with inputs and outputs
- Overview of the solution (architecture, systems to be changed and new IT-capabilities to be developed). This is added in steps; starting from an overview and getting detailed at the design stage.
- Quality requirements for 11 systems and services that are either new or are affected; detailing availability, capacity, peak time and response time for each.
- Requirements for test including test data and types of tests to be performed to verify the functionality and its quality.

The state of quality of existing applications or services is reviewed using the baseline requirements. Table 2 shows an excerpt of a table that documents the state for a main application. Maintenance teams have gained a lot of expertise on quality requirements. Before introducing the processes, the general feedback from these teams indicated loss of control because of the changes that are performed by projects and are not properly documented. Projects on the other hand complained that there is no documented overview of the “as-is” state (of course there is a lot of tacit knowledge) and they have to spend resources on gaining such overview in order to identify what should be changed and estimate the required effort.

Business units’ awareness of the importance of stating measurable quality requirements have increased as a result of work on specific projects. Quality requirements from a user point of view for different types of IT-capabilities are collected by business units and documented on the architecture portal. The list includes 37 concrete requirements and 15 requirements that should be specified for each new IT-capability. Types of IT-capabilities reflect

applications in our domain such as “decision-making service”, “online service” and “internal work surface”. Table 3 shows two requirements for online services; one requirement that applies to all projects (General) and one that should be further specified by projects.

Finally, projects use the above lists as input while they should identify solution-specific requirements as well. Here we present an example of an IT-capability delivered in 2015. The new service replaces letters sent by mail with digital letters and is of high importance for NAV as well as politicians who have defined the vision of a digital Norway. Examples of solution-specific requirements are:

- *Traceability* is important for NAV as well as for politicians in case of violations of security and privacy. It should be possible to trace the route of a digital letter from its origin to reception by end user. For verification, trace logs must be inspected at all test levels. This quality requirement is related to the ease of maintenance.
- Another example is *co-existence* with specific systems that produce the letters. Verification is done by integration testing.
- Business units require that letters can be digitalized *independently* of each other so that new letters can be added incrementally. Verification is performed by inspecting the design.
- *Capacity* of the service must be high enough to incorporate new letters.

Another example of the outputs of a requirement workshop is presented later in Section 7. Since 2014, all initiated projects and major deliveries document their quality requirements (at least 6 major deliveries each year and over 20 large projects), and even a special test team is established with focus on testing quality requirements during user acceptance testing. The impact on product quality cannot be measured for single products but we have moved from a state of having mostly tacit requirements to SMART requirements.

7. Experiences from requirement elicitation workshops

After developing the classification model explained in Section 4, we started a series of workshops in order to review existing quality requirements and to specify new requirements for the IT-capabilities to be developed in the modernization project.

The first workshop was conducted in March 2013 with focus on performance attributes of an online service for applying for benefits. We invited domain experts from business units, maintenance staff from the IT-department, a business domain architect and a software architect to the workshop. The authors of this paper participated in the workshop as well; one person as facilitator and another for documentation of the results and taking notes.

We initially planned to follow ATAM as the method for identifying the most important requirements as well as relating them to architectural decisions. Inputs to the ATAM method are business drivers, current architecture and architectural approaches. Output should be scenarios and risks that drive architectural decisions.

After explaining the method to the participants, we brainstormed for scenarios and high-level quality requirements. Ten important quality requirements were identified at this stage but no useful scenarios. We conducted two other workshops with focus on other business domains and had the same experience; i.e., the workshops helped in achieving a common understanding on top priority quality requirements from a business perspective while they were not successful in assessing the architectural impact of requirements or identifying detailed acceptance criteria.

We soon realized that although ATAM has many useful concepts for us, a full-fledged process was “overkill” for our situation. We observed the following problems:

- Participants from business units had very little experience in working with quality requirements.
- Participants were not familiar with ATAM and experienced it as heavy and complex.
- Software architects were unable to present important tradeoffs, sensitivity points or risks at this time. This was partly because of their inexperience with the method, and partly because of immaturity of architecture plans.

Other research has described similar challenges with the full-fledged ATAM approach and has suggested lightweight approaches [40]. After analyzing our observations and feedback from participants, we changed strategy. Instead of a full-fledged ATAM approach, we decided to focus on getting the most out of the workshops with as little formalism as possible. We consequently focused on *Business Drivers*, *Quality attributes* and *Scenarios*, not the other parts of ATAM. We also learned that it was better to have separate workshops with domain experts and technical experts. Since the QR-project team members (and afterwards software architects) act as facilitators in workshops, they balance stakeholder interests and document the results. The modified approach is more in the line of QAW [5], and greatly resembles what Chaparro and Keeling explain in their presentation called “The Mini-QAW” [10]. The approach is as follows:

Preparation step:

1. Analyze existing quality requirements and the state of quality for relevant IT-capabilities.
2. Identify the main stakeholders for the IT-capability to be developed or modified.
3. Invite the identified stakeholders to workshops of 2–3 hours’ duration.

Workshop with business domain experts:

4. Identify and discuss business goals that are important drivers of quality.
5. Introduce the existing state of services or applications and discuss the desired quality after the project.
6. Identify the most important quality attributes and requirements (as detailed as possible) and acceptance criteria using a suitable workshop technique. We used Impact Mapping [2] because it focuses on business goals and stakeholders but QAW steps 3.5 to 3.7 should also work perfectly. Terminology used in the workshops should be understandable for the participants and the focus should be on the quality of applications and services as experienced by end users.
7. Analyze the results offline, classify the identified quality requirements (or break down if too high level) and define possible follow up questions.
8. Inform the participants of the results. Collect and discuss feedback in order to have an approved list of prioritized quality requirements from a business point of view.

Workshop with technical experts: business domain experts often express quality requirements as “must have”. Impacts and costs are not fully evaluated and business domain experts have often limited information on technical risks. Additionally, business domain experts focus most on qualities observed by the end users. This means that more roles should contribute to make a complete set of quality requirements. Therefore, successive workshops with technical experts (software architects, operation architects, security experts, maintenance staff and developers) should be performed in order to:

9. Specify requirements for areas of concern not covered by previous workshops, such as security and maintainability.
10. Evaluate the consequences of identified requirements on existing applications, software architecture and IT-operations.

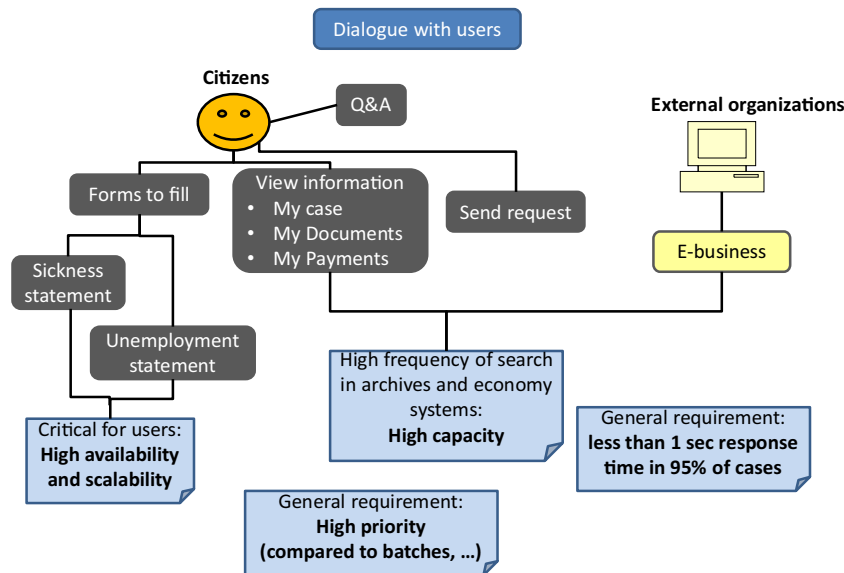


Fig. 6. Example of quality requirements for dialogue with end users.

11. Evaluate whether the identified requirements are attainable and are not in conflict with roadmaps and architecture principles.

Document and iterate:

12. Update the requirement list and ask for final feedback from all stakeholders. Conflicts and risks should be identified and discussed.

The key lessons are to separate the requirement elicitation activity from a business perspective from evaluating the impacts and detailing the technical requirements, starting from the existing state and not from scratch whenever possible, performing successive sessions and escalating in case of major disagreements. Scenarios are often discovered when discussing acceptance criteria. For some requirements we had to give clear feedback to the stakeholders on the cost, technical complexity or attainability and ask them to reevaluate the requirement.

There are also more mundane reasons for separate workshops. Participants from business units have often tight schedules and may be involved in several projects at the same time; thus they prefer short workshops. Additionally, in a large organization such as NAV there are many stakeholders that should be involved in the elicitation phase while workshops are most effective when the number of participants is low. However, the most important reason is the fact that an iterative process improved the *Establish* and *Detail* phases described in Section 5. After sending the results of workshops between groups, they can reevaluate their requirements or their assessments of the priority of a requirement. The drawback is the extra effort spent on communication but this in turn has improved the knowledge of the facilitators on quality requirements. The facilitator role is now performed by the members of the software architecture team who are also responsible for maintaining the basic lists.

We experienced that participants in the workshops learned the classification model gradually by applying it on their cases and has been using it as input to the subsequent meetings. Our method has been used extensively and we have performed between 15 and 20 such workshops in the last two years.

Fig. 6 shows a simplified example of the output of a workshop with business domain experts. The workshop focused on capturing quality requirements for NAV dialogue with end users. End users in this case are citizens as well as external organizations that

exchange information with NAV. The dialog with citizens happens in multiple channels: they may ask questions online using a chat line (Q&A), fill forms to apply for benefits, view information that NAV has collected on them, or send a request for a meeting. External organizations exchange data with NAV system-to-system (E-business). Requirements with the highest priority for each channel are identified and related to usage scenarios. Two general quality requirements are stated in the figure that stem from baseline requirements; i.e., “Online services have higher priority than batches during daytime” and “In 95% of interactions with end users, response time should be less than 1 second”. Other requirements are solution-specific; such as high availability of forms and high capacity for viewing information so that many users can be online at the same time. The figure does not include detailed descriptions of the identified quality requirements.

How business interacts with IT has a significant impact on innovation, the need for coordination and processes for decision making and change management. Our method fits to the existing organizational model with separate units responsible for business and IT development. There will always however be need for negotiation and tradeoffs across organizational units, functional areas and projects even if the organizational boundaries are changed.

We learned that for effective work with requirements:

1. Duration of a requirement elicitation workshop should not exceed 3 hours.
2. The number of participants in a workshop should not exceed 10.
3. Graphical presentations help summarizing and communication of the results to a high degree.
4. It is important to use collaboration technology that is easy to access for all such as internal wiki, and not word documents.
5. The workshop facilitator should consider different cultures and maturity of organizational units in working with requirements.
6. Future projects should have access to results of previous projects. Requirements should be well documented and easily accessible for all.

The architecture team has the following roles in the process:

- Managing processes for work with quality requirements

- Providing expertise to product owners by evaluating how proposed changes impact existing IT-capabilities and outlining architectural alternatives with their costs and risks
- Assisting projects to move from high-level quality attributes to detailed and application-based quality requirements. This is analogous to Maiden's discussion on user requirements vs. system requirements [30].
- Coordinating quality requirements across projects and stakeholders when conflicts are detected
- Providing architecture guidelines for projects
- Facilitating workshops, giving training and providing support in documentation
- Updating the architecture documentation gradually to reflect choices and decisions
- Maintaining the basic lists.

The architecture team is now much more mature in evaluating the consequences of quality requirements and documenting the background information. Business goals as expressed by product owners are the forces that significantly impact decisions. However, the business side needs support to evaluate costs, complexity and risks associated with quality requirements. The organization manages a lot of legacy systems and has defined roadmaps for increasing the quality of these systems in order to support business strategy and emerging needs. Whenever there is a risk that existing applications cannot fulfil a quality requirement or even act as barriers for the desired quality of a service, the risk is documented and added as action points for future projects or maintenance activities.

8. Discussion

This section positions the results described in this paper against related work and presents some key lessons.

8.1. Elicitation of quality requirements

We performed requirement workshops with stakeholders combined with pre-analysis, post-detailing and collecting feedback from stakeholders. Projects may apply any other method for requirement elicitation that they find suitable as well, including some lightweight approaches proposed for incremental development; such as those proposed in [5,12], as long as they support the *Establish* and *Detail* phases of the process described before.

In conducting requirement workshops, we were inspired by the Impact Mapping technique [2]. Impact mapping deals with *Actors*; i.e. who will be impacted by an outcome. In our approach we have used the term *Stakeholders* as those that will be impacted by the delivered product, either positively or negatively. The second level of an impact map sets the actors in the perspective of business goals and the *Impact* that will be created. In our approach the various quality characteristics represent the desired impact. The third level of an impact map focuses on what must be done to support the required impact; i.e. *Deliverables*. In a quality model, deliverables are quality sub-characteristics that should be realized to reach the desired impact. The resulting Impact Map is the summary of a workshop. Gojko Adzic, the creator of Impact Mapping, discusses typical mistakes done when applying this technique such as too many people, wrong people, criticizing too early, reverse engineering (knowing the desired output and rejecting other options), and going into too much low-level detail early [2]. These are warnings that we agree with.

When creating and detailing quality requirements, a common challenge is to specify measurable requirements accompanied with verification scenarios. Involving stakeholders such as testers (especially those who perform user-acceptance tests) and maintenance

experts in the process of requirement elicitation is important to help identifying the verification scenarios. In NAV, the quality requirement test team has gained lots of expertise on acceptance testing. Reusing lists of requirements facilitates the requirement elicitation process as well, while some requirements must be tailored to specific project contexts. Doer et al. call such lists for *experience-based artefacts* [16].

Another challenge is communication of requirements and knowledge reuse [42]. Loss of knowledge in product handover between projects and maintenance staff or contractors is a common problem. Long-living applications, incremental development and staff turnovers call for a systematic method for maintaining and reusing knowledge about product quality. Collecting a manageable number of quality requirements and organizing them in a simple and accessible overview facilitates reuse and knowledge sharing. Relationship maps are already in use for functional requirements in some projects and it would be of interest to develop such maps for important quality requirements as well.

When we compare quality requirements in our context with the most frequent requirements mentioned in [7], we observe some similar characteristics and some differences. Frequent changes to legislations and policy rules should be implemented quickly and cost-effectively, thus modifiability of applications is of high importance, as also reported in [7]. In addition, since IT-capabilities are used by a large number of citizens in different stages of their lives, usability is critical. NAV handles social benefit payments and therefore availability and security are essential characteristics. Interestingly, security is not one of the requirements with highest frequency in [7].

8.2. Classification of quality requirements

The ISO/IEC 25010 standard was selected in NAV for the classification of requirements since it was widely known in the organization. A contribution of the work described in this paper has been tailoring the standard to the context of the organization by modifying some characteristics and providing specific quality characteristics at the third level based on dialogue with requirement owners. In [29] for example, the third level is used for formulating and organizing quality requirements that relate to code quality. Other examples and approaches to tailoring are discussed in [27]; for example, by defining goals.

8.3. Agile development and maintenance

SAFe's proposed handling of NFRs has a good amount of overlap with the result of our work. In fact, the whole software development process at NAV has much resemblance to SAFe. However, when the authors started working with quality requirements at NAV, SAFe was in a much earlier stage and less known throughout the community. Because of this we were not influenced by this framework during our work but we are happy to see overlap and resemblance on a conceptual layer.

IT-modernization is a continuous state in NAV and not just a temporary phase. In our experience modernization is often a combination of reuse and new development. This is in line with Basili's opinion that reuse can be considered as an improvement paradigm that helps organizations evaluate, learn and enhance their software processes and products [6]. In NAV, we see that the focus on product quality has particular utility in the maintenance of a complex IT environment. In [38], Veld and Krogstie report that Norwegian organizations spend almost 41% of their effort on maintenance activities (corrective, adaptive, enhance and non-functional perfective maintenance), while they spend 17% on replacement and new development. The remaining 42% of effort is spent on technical operation and user support. The authors also

show that these ratios have been stable over time regardless of innovations in methodology and technology. IT applications in NAV are long-lived and they need to be evolvable and maintainable for many years. Our results show the importance of having methodologies that can fit the maintenance state as well, not just green field development. The methodology and processes discussed in this article are developed with this in mind.

8.4. Validity of the results

Empirical research is concerned with observations and interpretations. The goal of an empirical study is to have an improved and evidenced-based theory or practice. Empirical studies may be performed quantitatively, qualitatively or in combination [13]. The primary intent of qualitative studies (such as case studies, interviews with open-ended questions or observations) is to generate rich and often context-specific understanding. Qualitative studies often take place in the natural settings and qualitative data is usually subjective, unstructured and non-numeric. There are techniques to improve the validity of findings of a qualitative study such as using multiple sources in data collection, having key informants to review the results, and using theory in research design [13,41].

Earlier research has proposed some approaches for evaluating new IT development methodologies and tools [32]. Examples are performing a cost-benefit or Return-On-Investment analysis, or performing surveys using evaluation criteria such as those proposed by the *Technology Acceptance Model (TAM)* [14]. TAM; originally developed by Davis; is a generic model which can be applied to measure the acceptance and infusion of a technology or information system. There are several extensions of TAM that combine it with other models. One such model is proposed by Dybå et al. [18] and was applied for evaluating Model-Driven Engineering (MDE) in four companies [33]. The study showed that although the survey collected perceptions of the participants on MDE, several data sources should be combined to reach valid conclusions (for example, combining survey with observations), the results are still case specific and the evaluation should be performed over time.

This work presents experiences on improving software development process within a specific context to address concerns on the quality of developed products. During the work, several strategies were applied to ensure that the classification model and processes are useful and provide actual improvements for the organization:

- We involved business experts as well as technical experts in developing the quality requirements' classification model.
- The processes have been reviewed multiple times by internal experts.
- Feedback and collected material from workshops have influenced the outcomes.

The quality requirement classification model and processes are integrated in the agile development process and are adopted by the organization. The ease of use has always been in mind by adding examples, making the requirement lists available for reuse, and simplifying the processes. Prior knowledge of the organization and valuable feedback from colleagues has improved our confidence in the results. However, the viewpoint of evaluation is qualitative and based on opinions of the practitioners. One action that could reinforce the finding in this paper is to perform surveys at some point in the future and combine it with the current observational data.

We have contributed to the body of knowledge by taking into account the impact of size (large organization and many stakeholders), the impact of legacy systems, combining new development with continuous maintenance, and considering the issue of technical debt. An experience report is often not concerned with the generalization of methods and practices; rather it assesses

"usefulness" in the current and similar contexts. Many of our observations match experiences of others, as discussed in Sections 8.1 to 8.3. We have applied the methods in several large projects and therefore argue that the methods are reusable in the context of large organizations and projects where agile development is practiced. Whether the outcomes are applicable in other contexts is however difficult to prove and other practitioners must evaluate the usefulness of our findings in their settings. Several conference talks have come out of our work and we have shared experience, techniques and tools with other public sector organizations in Norway. In addition, one of the co-authors has reused some of results of this work, such as the requirement elicitation workshops, in other organizations. This indicates general applicability of proposed methods and techniques.

8.5. Key lessons from our work

Many factors raise the complexity of managing quality requirements in a large organization. Some key lessons for successful work with quality requirements are summarized here:

- In a large organization, the requirement owner's role is important. A business or IT unit should be responsible for managing the state of requirements for a domain, functional area or applications. The requirement owner should define a vision for the quality in the domain of interest and refine it iteratively in negotiation with other stakeholders.
- Often no single entity can specify requirements for an entire IT-capability except for minor modifications. In [24], Tom Gilb and Kai Gilb discuss the complexity of the product owner role in large projects. The same apply to large organizations.
- Established agile methodologies such as Scrum have little guidelines on how to manage quality requirements. Short iterations and continuous delivery of functionality may lead to neglecting or under-emphasizing product quality. We have added steps and guidelines in the development process specific to quality requirements. Single agile teams often do not have expertise on complete value chains and software architects must be involved to keep an overview.
- When multiple development teams or projects modify the same applications, development plans should be coordinated to achieve the desired quality. Agile development teams should be able to change scope of the delivery, delivery plans and the prioritization of backlog items. However, applying agile methods to large projects creates challenges. Fulfillment of quality requirements may cause conflicts between interests of a team (and their priorities) and requirement owners, or between teams and between projects. Sometimes the disagreement needs to be escalated to higher level management.
- The NAV IT-landscape is complex, consisting of old and new systems of different types, developed on multiple technologies and with different groups of users. The quality of a new service often depends on the quality of a chain of underlying applications. Therefore, it has been critical to establish a "status-quo" of existing applications. This is a complex and demanding task that must be done incrementally and continuously as applications change. The status-quo acts also as input for identifying technical debt and is used in negotiation with product owners and developers / contractors.
- Evaluating the cost of fulfilling a quality requirement is often challenging. For example, improving availability of a service may require extending availability of underlying services or having access to help desk and support staff for an extended time period. Since we started the QR-project, the awareness

of business side on the investment needed to improve quality and the infrastructure supporting it has increased significantly. The result is better management support for initiatives upgrading applications and infrastructure.

- The increased awareness on improving quality has led NAV to develop a dedicated process for managing technical debt. Unfulfilled requirements are added to the portfolio backlog as technical debt, and then monitored and evaluated in future development. Presenting the whole method is out of the scope of this paper.
- Improving quality must be done incrementally and requires a roadmap that is approved by higher-level management. Often the budget for improving quality (and removing technical debt) is limited and costs should be spread over time and multiple projects. As a result of the gained awareness on quality requirements, new projects are required to explicitly include costs of required modification of existing applications in their budgets, not relying on the normal maintenance loops.

Finally, we must emphasize the importance of involving the business side in the work for specifying quality requirements, prioritizing, understanding costs drivers and making tradeoffs. The quality requirement management process is highly incremental and iterative; i.e., analyzing the current state, improving quality in steps, revising the state and communicating it continuously to the organization. The work requires highly motivated staff, with domain and architecture expertise to monitor and assist ongoing work in projects.

9. Conclusions and future work

This paper presents the quality requirements management practice in a large public organization after nearly 3 years of work. Our practice is the result of evolving and combining several methodologies, techniques and industry practices. Our extended quality classification model, workshop techniques and the overall processes have now been in use for some time. Our work now focuses on defining strategic goals and roadmaps for software quality, managing the quality requirement lists, highlighting the gaps, and supporting projects for setting measurable quality objectives. In addition, we continuously work on refining the methodology and processes so that NAV is capable of managing product quality in an efficient manner. In particular, we see that process documentation needs simplification to foster better understanding by business units.

One of the main obstacles of our work is the complexity of the IT-landscape combined with both functional and technological diversity. There is also continuous pressure on the organization to deliver new functionality while keeping maintenance cost under control. NAV is currently increasing its participation in the development phase as a means to get better control of costs as well as quality. Further we see that focus on agile software development has potential adverse effects on quality if functionality is always prioritized without paying enough consideration to quality.

The main contribution of this work to the IT community is incorporating many techniques and methodologies into a complete framework that covers the whole software development lifecycle; from new development to maintenance, and in both greenfield and brownfield projects. Almost no development starts from scratch in our context and probably in most other large organizations as well. We have therefore developed processes that combine elicitation of new quality requirements with mandatory requirements and the quality state of legacy applications. The classification model and basic lists act as elicitation guidelines and are actively used. We have combined existing techniques such as QAW and

Impact Mapping to create a lightweight, iterative and effective quality requirement elicitation process that cultivated ownership, shared responsibility and understanding of quality requirements. In other words, we have gone from theory to practice in a very large and complex environment and have highlighted challenges of working with multiple projects running in parallel and a large number of stakeholders. We have demonstrated practical use of the ISO/IEC-25010:2011 classification model by tailoring it to fit the context and our model has received positive feedback since tailoring made the standard more usable and specific.

We consider the work with product quality management successful in achieving its goals. A factor contributing to its success has been the support given by management to increase focus on quality. An advantageous effect of our work is that the organization now has a known software product quality state for a large number of applications and services. Having an overview of the “as-is” state helps avoiding ad-hoc prioritizations. In addition, requirement negotiation between developers and requirement owners in the elicitation phase helps avoiding disagreements and conflicts down the line. A second motivation is reducing technical debt, which could be a significant problem if not managed. Finally, implementing the right quality at the right time is very important since quality cannot always be retrofitted into the software, and with insufficient or wrong qualities the software can become unfit for its business purpose. Having the ability to communicate quality requirements to the organization, contractors and even to politicians fosters realistic expectations. For NAV, this work has caused a major increase in maturity and consciousness with regards to software product quality management and IT projects are governed better with regards to quality, as well as scope, cost and time.

Acknowledgment

We are grateful to NAV for supporting the publication of the paper. We are also grateful to Dr. Jason Baragry for his valuable comments during the writing of this paper.

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] P. Abrahamsson, M. Babar, P. Kruchten, *Agility and architecture: can they co-exist?* IEEE Softw. 27 (2) (2010) 16–22.
- [2] G. Adzic, *Impact Mapping, Making a Big Impact with Software Products and Projects*, 2012 ISBN 978-0-9556836-4-0.
- [3] S. Ambler, M. Lines, *Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise*, IBM Press, 2012.
- [4] ATAM, <http://www.sei.cmu.edu/architecture/tools/evaluate/atam.cfm> (accessed in October 2014).
- [5] M.R. Barbacci, R.J. Ellison, A.J. Lattanze, J.A. Stafford, C.B. Weinstock, W.G. Wood, *Quality Attribute Workshops (QAWs)*, Third Edition, Carnegie Mellon University, 2003.
- [6] V.R. Basili, *Viewing maintenance as reuse-oriented software development*, IEEE Softw. 7 (January(1)) (1990) 19–25.
- [7] S. Bellomo, I. Gorton, R. Kazman, *Towards agile architecture, insights from 15 years of ATAM data*, IEEE Softw. 32 (5) (2015) 38–45.
- [8] J.D. Blaine, J. Cleland-Huang, *Software quality requirements: how to balance competing priorities*, IEEE Softw. 25 (2) (2008) 22–24.
- [9] B.W. Boehm, *Characteristics of Software Quality*, North-Holland, 1978.
- [10] W. Chaparro, M. Keeling, *Facilitating the Mini-Quality Attributes Workshop*, Presentation at SATURN, 2014 available online at <http://resources.sei.cmu.edu/library> (accessed in May 2016).
- [11] L. Chen, M.A. Babar, B. Nuseibeh, *Characterizing architecturally significant requirements*, IEEE Softw. 30 (2) (2013) 38–45.
- [12] J. Cleland-Huang, A. Czauderna, E.D. Keenan, *A persona-based approach for exploring architecturally significant requirements in agile projects*, in: *Proceedings of REFSQ 2013*, 2013, pp. 18–33.
- [13] J.W. Creswell, *Research Design, Qualitative, Quantitative, and Mixed Methods Approaches*, Sage Publications, 2002.
- [14] F. Davis, *Perceived usefulness, perceived ease of use, and user acceptance of information technology*, MIS Quart. 13 (3) (1989) 318–339.
- [15] *Disciplined Agile 2.0*; 2015 <http://www.disciplinedagiledelivery.com> (accessed in August 2016).

- [16] J. Doerr, D. Kerkow, T. Koenig, T. Olsson, Non-functional requirements in industry – Three case studies adopting an experience-based NFR method, in: *Proceedings of Intl. IEEE Conference on Requirements Engineering*, 2005, pp. 373–384.
- [17] G.T. Doran, There's a S.M.A.R.T. way to write management's goals and objectives, *Manag. Rev.* 70 (11) (1981) 35–36 (AMA FORUM).
- [18] T. Dybå, N.B. Moe, E.M. Mikkelsen, An empirical investigation on factors affecting software development acceptance and utilization of electronic process guides, in: *Proceedings of the 10th International Symposium on Software Metrics (Metrics'04)*, 2004, pp. 220–231.
- [19] Extreme Programming, 2009 www.extremeprogramming.org (accessed in August 2016).
- [20] Enterprise Unified Process, 2013 <http://www.enterpriseunifiedprocess.com> (accessed in August 2016).
- [21] X. Franch, J.P. Carvallo, Using quality models in software package selection, *IEEE Softw.* 20 (1) (2003) 34–41.
- [22] T. Gilb, *Principles of Software Engineering Management*, Addison-Wesley, 1998 ISBN 0-201-19246-2.
- [23] T. Gilb, *Competitive Engineering: A Handbook for Systems & Software Engineering using Planguage*, 2005 ISBN 0-7506-6507-6.
- [24] T. Gilb, K. Gilb, Advanced product owners, *Agile Rec.* (17) (2014) Feb..
- [25] H. In, B. Boehm, T. Rodgers, M. Deutsch, Applying winwin to quality requirements: a case study, in: *Proceedings of the 23rd International Conference on Software Engineering (ICSE 2001)*, IEEE Press, 2001, pp. 555–564.
- [26] ISO/IEC 25010, *Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*, 2011 <http://www.iso.org>.
- [27] M. Klas, C. Lampasona, J. Munch, Adapting software quality models: practical challenges, approach, and first empirical results, in: *Proceedings of the 37th EUROMICRO Conference on Software Engineering and Advanced Applications*, IEEE Computer Society, 2011, pp. 341–348.
- [28] LESS; <https://less.works> (accessed in August 2015).
- [29] J.L. Letouzey, The SQALE method for evaluating technical debt, in: *Proceedings of the 3rd International Workshop on Managing Technical Debt (MDT'12)*, IEEE Press, 2012, pp. 31–36.
- [30] N. Maiden, User requirements and system requirements, *IEEE Softw.* 25 (2) (2008) 90–91.
- [31] J.P. Miguel, D. Mauricio, G. Rodríguez, A review of software quality models for the evaluation of software products, *Int. J. Softw. Eng. Appl.* 5 (6) (2014) 31–53.
- [32] P. Mohagheghi, Evaluating software development methodologies based on their practices and promises, in: *Proceedings of SoMet 2008 (New Trends in Software Methodologies, Tools and Technique)*, 2008, p. 14.
- [33] P. Mohagheghi, An approach for empirical evaluation of model-driven engineering in multiple dimensions, in: *Proceedings of 5th C2M: EEMDD Workshop at European Conference on Modelling Foundations and Applications (ECMFA)*, 2010, pp. 6–17.
- [34] SAFe; <http://www.scaledagileframework.com> (accessed in August 2016).
- [35] Scrum; <http://www.scrum.org> and <http://www.scrumguides.org> (accessed in April 2015).
- [36] I. Sommerville, P. Sawyer, *Requirements Engineering: A Good Practice Guide*, John Wiley & Sons, Great Britain, 1997.
- [37] E. Tom, A. Aurum, R. Vidgen, An exploration of technical debt, *J. Syst. Softw.* 86 (6) (2013) 1498–1516.
- [38] T.K. Veld, J. Krogstie, Work distribution in Information systems development, maintenance and evolution, in: *Proceedings of NOKOBIT conference*, 22, 2014.
- [39] S. Wagner, K. Lochmann, L. Heinemann, M. Kläs, A. Trendowicz, R. Plösch, A. Seidl, A. Goeb, J. Streit, The quamoco product quality modelling and assessment approach, in: *Proceedings of the 34th International Conference on Software Engineering, ICSE 2012*, IEEE Press, 2012, pp. 1133–1142.
- [40] E. Woods, Industrial architectural assessment using TARA, in: *Proceedings of 9th IEEE/IFIP Conference on Software Architecture (WICSA)*, 2011, pp. 56–65.
- [41] R.K. Yin, *Case Study Research, Design and Methods*, Sage Publications, 2002.
- [42] D. Zowghi, C. Coulin, *Requirements elicitation: a survey of techniques, approaches, and tools*, in: *Engineering and Managing Software Requirements*, Springer, Berlin Heidelberg, 2005, pp. 19–46.